

First 1.8-inch hard drive

Toshiba has revealed the industry's first 1.8-inch microSATA based hard drives for PMPs and tablets

Do Not Track

Google and Mozilla have announced a 'Do Not Track' list that allows users to opt out of having their activities tracked by online advertisers

Developer corner

```
var word_list = {
  "Mah" : "My",
  "Puhleaze" : "I am an idiot",
  "CWG" : "Common Wealth Games",
  "FT" : "FastTrack",
  "": ""};
//This is to indicate that the
word_list has ended.
//Notice the comma after each
entry but the last one. Also
notice the
//semicolon after the curly paren-
thesis. C++ Structures anyone?
String.prototype.prepareRegex =
function()
{
  return this.replace(/\[\\
  [\\^\&\$\\.\\(\)\?\\\/\\+\\{\}\\\]/g,
  "\\$1");
//this is keyword signifying the
current object, exactly like in
Python.
//Regular Expression is not
explained, as it will worthy
another tutorial.
//Reading an one page online tuto-
rial is recommended
};
// To ensure word replacement
doesn't happen in source code
under tags like link etc.
function isOkTag(tag)
{
  return ("pre,blockquote,co
  de,input,button,textarea".
  indexOf(", "+tag) == -1);
}
var regexs=new
Array(),replacements=new Array();
//Searching for Matching words
which needs to be replaced.
//push and pop method provide
stack type access. Push inserts
and Pop returns the
//items pushed in.
for(var word in word_list) //Loop
for every word in word_list
{
  if(word != "") //Remember the
  last word was a blank in word_list
  {
    regexs.push(new RegExp("\\b"+word.
    prepareRegex().replace(/\*/g, '^
    ]*')+ "\\b", 'gi'));
    replacements.push(word_
    list[word]);
  }
}
//Evaluate method results in a
XpathResult object, the '6' here
in the parameter list indicates
//the output format.
var texts = document.eval-
uate("//*[text() [normalize-
space(.)!='']]", document.
body,null,6,null), text="";
```

```
//Notice closely, the for is ini-
tializing two variables instead of
the usual one.
//snapshotitem method provides
access to matching nodes, modifi-
cation of 'document' doesn't
//invalidate the collection. 'I'
is the index passed, specifying
the position of matching node.
//'replace' is method of the
string class performing the func-
tion of replacing first
//argument by second argument.
//similarly tolowercase makes the
match case insensitive.
for (var i=0,l=texts.snapshot-
Length; (this_text=texts.
snapshotItem(i)); i++)
{
  if(isOkTag(this_text.parent-
Node.tagName.toLowerCase()) &&
  (text=this_text.textContent))
  {
    for(var x=0,l=regexs.length; x<l;
    x++)
    {
      text = text.replace(regexs[x],
      replacements[x]);
      this_text.textContent = text;
    }
  }
  //Remember to not use a word
  processor, as it may convert your
  double-quote marks into smart-
  quotes.
```

The above script demonstrates to a small extent the power that Greasemonkey bestows on the end-user. As a final hack, we will learn how to embed an image into the source code itself. Yes by that we mean representing an image in text. The benefit of doing that is obvious. If you want to insert an image

anywhere via a Greasemonkey script, in the conventional manner you would be putting insane load on the server hosting that image, since every time someone loads the image, that image will need to be fetched from the server and displayed. This may also increase page loading time. This problem arises of a fundamental limitation of Greasemonkey scripting because it does not natively provide any way to bundle resource data like photographs, audio etc along with the script, but then expecting greasemonkey scripting to substitute an entire language might be a little too much. So here is how you go around it.

Instead of posting the image to a web server and having your user script fetch it, you can embed the image data in the script itself by using data: URL. A data: URL allows you to encode an image as printable text, so you can store it as a JavaScript string. And Firefox supports data: URLs natively, so you can insert the graphic directly into a web page by setting an img element's src attribute to the data: URL string. Firefox will display the image without sending a separate request to any remote server. You can construct data: URLs from your own image files at <http://software.hixie.ch/utilities/cgi/data/data>.

What you can do with GreaseMonkey scripting is limited only by your imagination and your set of technical know-how. This was very brief introduction to the world of GreaseMonkey scripting. Happy Personalisation. 📄



Flickr.com before and after...

